EF Migration

Thursday, April 2, 2026 2:53 PM

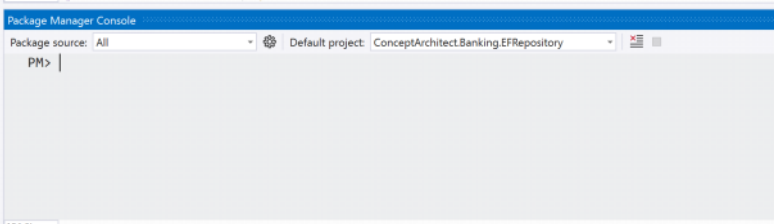
We need to install EFTools for EF Migration ("We have already done that")

What is Migration?

- It is a snapshot of our current Object Model
- The migration helps us know
  - What is present in OO
  - How is it going to be mapped to database

1. Option Package Manager Console

- Make sure your Default project is one
  - Which contains your context
  - Where you want to maintain the migration s

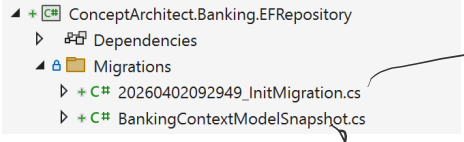


2. Add the First Migration

```
PM> Add-Migration InitMigration
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

- Note:
  - Migration Name should be something that can be used a class name
  - It internally will create a class

What just happened




```
[DbContext(typeof(BankingContext))]
0 references
partial class BankingContextModelSnapshot : ModelSnapshot
{
    0 references
    protected override void BuildModel(ModelBuilder modelBuilder)
    {
#pragma warning disable 612, 618
        modelBuilder
            .HasAnnotation("ProductVersion", "9.0.14")
            .HasAnnotation("Relational:MaxIdentifierLength", 128);

        SqlServerModelBuilderExtensions.UseIdentityColumns(modelBuilder);
#pragma warning restore 612, 618
    }
}
```

- Creates a snapshot of all objects that should be saved.
- It is description of our current object and their properties

- Suggests changes required in the database
  - Up suggest what will change if the migration is applied
    - Currently nothing
  - Down suggests what will change if this migration is removed.

```
/// <inheritdoc />
1 reference
public partial class InitMigration : Migration
{
    /// <inheritdoc />
    0 references
    protected override void Up(MigrationBuilder migrationBuilder)
    {

    }

    /// <inheritdoc />
    0 references
    protected override void Down(MigrationBuilder migrationBuilder)
    {

    }
}
```

Everytime you change anything in your Data model you should create a new migration

- So let us add our Customer in the Db Context

```
public class Customer : Entity<string>
{
    0 references
    public string Name { get; set; }
    5 references
    public string Id { get; set; }  //email

    0 references
    public string Address { get; set; }

    1 reference
    public bool IsActive { get; set; }

    2 references
    public string Password { get; set; }


}
```

```
2 references
public class BankingContext : DbContext
{
    string connectionString= "Data Source=(LocalDB)\\MSSQL
    0 references
    protected override void OnConfiguring(DbContextOptions

    0 references
    public DbSet<Customer> Customers { get; set; }

}
```

Let's now add a Migration

- A migration name should be descriptive like a test name or a commit

```
PM> Add-Migration CustomerAdded
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

```
Migrations
    20260402092949_InitMigration.cs
    20260402093754_CustomerAdded.cs
    BankingContextModelSnapshot.cs
```

- It creates a new Migration file and updates the model snapshot
- It doesn't update old Migration files like InitMigration

Model Snapshot

```
partial class BankingContextModelSnapshot : ModelSnapshot
{
    0 references
    protected override void BuildModel(ModelBuilder modelBuilder)
    {
#pragma warning disable 612, 618
        modelBuilder
            .HasAnnotation("ProductVersion", "9.0.14")
            .HasAnnotation("Relational:MaxIdentifierLength", 128);

        SqlServerModelBuilderExtensions.UseIdentityColumns(modelBuilder);
```

```
0 references
partial class BankingContextModelSnapshot : ModelSnapshot
{
    0 references
    protected override void BuildModel(ModelBuilder modelBuilder)
    {
#pragma warning disable 612, 618
        modelBuilder
            .HasAnnotation("ProductVersion", "9.0.14")
            .HasAnnotation("Relational:MaxIdentifierLength", 128);

        SqlServerModelBuilderExtensions.UseIdentityColumns(modelBuilder);

        modelBuilder.Entity("ConceptArchitect.Banking.Customer", b =>
            {
                b.Property<string>("Id")
                    .HasColumnType("nvarchar(450)");

                b.Property<string>("Address")
                    .IsRequired()
                    .HasColumnType("nvarchar(max)");

                b.Property<bool>("IsActive")
                    .HasColumnType("bit");

                b.Property<string>("Name")
                    .IsRequired()
                    .HasColumnType("nvarchar(max)");

                b.Property<string>("Password")
                    .IsRequired()
                    .HasColumnType("nvarchar(max)");

                b.HasKey("Id");

                b.ToTable("Customers");
            });
#pragma warning restore 612, 618
    }
}
```

C# Propr

DB Column

Table name

Primary Key

```
public partial class CustomerAdded : Migration
{
    /// <inheritdoc />
    0 references
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "Customers",
            columns: table => new
            {
                Id = table.Column<string>(type: "nvarchar(450)", nullable: false),
                Name = table.Column<string>(type: "nvarchar(max)", nullable: false),
                Address = table.Column<string>(type: "nvarchar(max)", nullable: false),
                IsActive = table.Column<bool>(type: "bit", nullable: false),
                Password = table.Column<string>(type: "nvarchar(max)", nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_Customers", x => x.Id);
            });
    }

    /// <inheritdoc />
    0 references
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropTable(
            name: "Customers");
    }
}
```

- When you apply this migration
  - I will create the table

- If you ever remove this migration
  - I will drop the table

Let's add a few fields into Customer

```
21 references
public class Customer : Entity<string>
{
    0 references
    public string Name { get; set; }
    5 references
    public string Id { get; set; }  //email

    0 references
    public string Address { get; set; }

    1 reference
    public bool IsActive { get; set; }

    2 references
    public string Password { get; set; }

    0 references
    public string Photo { get; set; }

    0 references
    public string Phone { get; set; }

}
```

```
PM> Add-Migration CustomerPropertiesAdded_Photo_Phone
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

```
1 reference
public partial class CustomerPropertiesAdded_Photo_Phone : Migration
{
    /// <inheritdoc />
    0 references
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AddColumn<string>(
            name: "Phone",
            table: "Customers",
            type: "nvarchar(max)",
            nullable: false,
            defaultValue: "");

        migrationBuilder.AddColumn<string>(
            name: "Photo",
            table: "Customers",
            type: "nvarchar(max)",
            nullable: false,
            defaultValue: "");
    }

    /// <inheritdoc />
    0 references
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.DropColumn(
            name: "Phone",
            table: "Customers");

        migrationBuilder.DropColumn(
            name: "Photo",
            table: "Customers");
    }
}
```

- All fields are by default required fields in EF Core
- If any field is optional you should mark it optional by adding "?"

Making Fields Optional

```
public class Customer : Entity<string>
{
    0 references
    public string Name { get; set; }
    5 references
    public string Id { get; set; }  //email

    0 references
    public string Address { get; set; }

    1 reference
    public bool IsActive { get; set; }

    2 references
    public string Password { get; set; }

    0 references
    public string? Photo { get; set; }

    0 references
```

```
1 reference
public partial class CustomerProperties_Modified_Photo_Phone_Optional : Migration
{
    /// <inheritdoc />
    0 references
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AlterColumn<string>(
            name: "Photo",
            table: "Customers",
            type: "nvarchar(max)",
            nullable: true,
            oldClrType: typeof(string),
            oldType: "nvarchar(max)");

        migrationBuilder.AlterColumn<string>(
            name: "Phone",
            table: "Customers",
            type: "nvarchar(max)",
            nullable: true,
```

```
    2 references
    public string Password { get; set; }

    0 references
    public string? Photo { get; set; }

    0 references
    public string? Phone { get; set; }

}
```

```
        migrationBuilder.AlterColumn<string>(
            name: "Phone",
            table: "Customers",
            type: "nvarchar(max)",
            nullable: true,
            oldClrType: typeof(string),
            oldType: "nvarchar(max)");
    }

    /// <inheritdoc />
    0 references
    protected override void Down(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.AlterColumn<string>(
            name: "Photo",
            table: "Customers",
            type: "nvarchar(max)",
            nullable: false,
            defaultValue: "",
            oldClrType: typeof(string),
            oldType: "nvarchar(max)",
            oldNullable: true);
```

REMOVING THE MIGRATION

- You may remove migrations but only one by one
- Remove-Migration will remove he latest Migration
- Let's remove all of them one by name

```
Build succeeded.
Removing migration '20260402094931_CustomerProperties_Modified_Photo_Phone_Optional'.
Reverting the model snapshot.
Done.
PM> Remove-Migration
Build started...
Build succeeded.
Removing migration '20260402094512_CustomerPropertiesAdded_Photo_Phone'.
Reverting the model snapshot.
Done.
PM> Remove-Migration
Build started...
Build succeeded.
Removing migration '20260402093754_CustomerAdded'.
Reverting the model snapshot.
Done.
PM> Remove-Migration
Build started...
Build succeeded.
Removing migration '20260402092949_InitMigration'.
Removing model snapshot.
Done.
PM>
```

Let's now apply the migration

```
PM> Add-Migration Customer_Added
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

Let's have Accounts Now

```
0 references
public  class BankAccount
{
    0 references
    public int AccountNumber { get; set; }
    2 references
    public double Balance { get; set; }

    0 references
    public void Debit(double amount)
    {
        Balance -= amount;
    }
    0 references
    public void Credit(double amount)
    {
        Balance += amount;
    }
}
```

- Let's add a DataSet

```
public class BankingContext : DbContext
{
    string connectionString= "Data Source=(LocalDB)\\MS
    0 references
    protected override void OnConfiguring(DbContextOpti

    0 references
    public DbSet<Customer> Customers { get; set; }
    0 references
    public DbSet<BankAccount> Accounts { get; set; }

}
```

Add Migration

```
PM> Add-Migration Customer_Added
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Add-Migration Accounts_Added
Build started...
Build succeeded.
Unable to create a 'DbContext' of type 'BankingContext'. The exception 'The entity type 'BankAccount' requires a primary key to be defined. If
you intended to use a keyless entity type, call 'HasNoKey' in 'OnModelCreating'. For more information on keyless entity types, see https://
go.microsoft.com/fwlink/?linkid=2141943.' was thrown while attempting to create an instance. For the different patterns supported at design time,
see https://go.microsoft.com/fwlink/?linkid=851728
```

What happened?

- Normally every entity is expected to have a primary key as per RDBMS
  - While It is NOT mandatory it is required

- If we do not have a primary key we must add the attribute
  - HasNoKey
  - But this is not our cases

- By default if your entity has property called "Id" or "ID" it is used as the primary key
  - We have a AccountNumber property that I want to use as Primary Key
    - But EF doesn't know that
    - We have to tell EF that I want AccountNumber as Primary Key

```
1 reference
public  class BankAccount
{
    [Key]
    0 references
    public int AccountNumber { get; set; }
    2 references
    public double Balance { get; set; }

    0 references
    public void Debit(double amount)
    {
        Balance -= amount;
    }
    0 references
    public void Credit(double amount)
    {
        Balance += amount;
    }
}
```

Define relationship

```
21 references
public class Customer
{
    0 references
    public string Name { get; set; }

    [Key]
    0 references
    public string Email { get; set; }  //email
```

```
2 references
public  class BankAccount
{
    [Key]
    0 references
    public int AccountNumber { get; set; }
    2 references
    public double Balance { get; set; }
```

```
    public string Name { get; set; }

    [Key]
    0 references
    public string Email { get; set; }  //email

    0 references
    public string Address { get; set; }

    1 reference
    public bool IsActive { get; set; }

    2 references
    public string Password { get; set; }

    0 references
    public string? Photo { get; set; }

    0 references
    public string? Phone { get; set; }

    0 references
    public virtual IList<BankAccount> Accounts { get; set; } = new List<BankAccount>();

}
```

```
public class BankAccount
{
    [Key]
    0 references
    public int AccountNumber { get; set; }
    2 references
    public double Balance { get; set; }

    //reference to Customer
    0 references
    public virtual Customer Owner { get; set; }

    0 references
    public void Debit(double amount)
    {
        Balance -= amount;
    }
    0 references
    public void Credit(double amount)
    {
        Balance += amount;
    }
}
```

Note: Mark these relationships as virtual

- It will help entity framework fetch them efficiently

```
PM> Add-Migration Customer_Account_Relationship_Added
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM>
```

```
/// <inheritdoc />
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.AddColumn<string>(
        name: "OwnerEmail",
        table: "Accounts",
        type: "nvarchar(450)",
        nullable: false,
        defaultValue: "");

    migrationBuilder.CreateIndex(
        name: "IX_Accounts_OwnerEmail",
        table: "Accounts",
        column: "OwnerEmail");

    migrationBuilder.AddForeignKey(
        name: "FK_Accounts_Customers_OwnerEmail",
        table: "Accounts",
        column: "OwnerEmail",
        principalTable: "Customers",
        principalColumn: "Email",
        onDelete: ReferentialAction.Cascade);
}
```

IMPORTANT: Migrations have not caused any changes to datbase

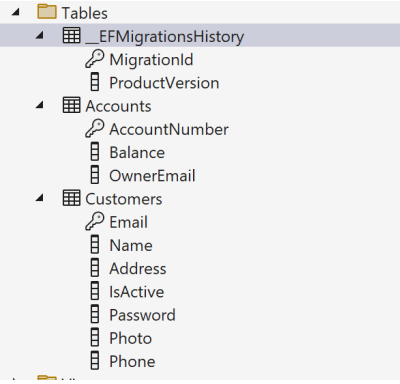
- Currently
  - No tables are created during the migration
- It is just a snapshot of changes required in databases.
  - But no changes are done yet

Now Let's update the Database

```
PM> Update-Database
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this takes too long.
Applying migration '20260402095310_Customer_Added'.
Applying migration '20260402100332_Customer_id_changed__Author_added'.
Applying migration '20260402100808_Customer_Account_Relationship_Added'.
Done.
PM>
```

Now it will generate tables

- It will generate tables for
  - Customers
  - Accounts
- But it will also generate table for Migration History



	MigrationId	ProductVersion
▶	20260402095310_Customer_Added	9.0.14
	20260402100332_Customer_id_changed__Author_added	9.0.14
	20260402100808_Customer_Account_Relationship_Added	9.0.14
*	*NULL*	*NULL*

This table Helps Update-Database to know what migrations are already applied and it applies new ones only

You can remove only those migrations that are not updated to database yet

```
PM> Remove-Migration
Build started...
Build succeeded.
Removing migration '20260402101639_BankAccount_Status_Added'.
Reverting the model snapshot.
Done.
PM> Remove-Migration
Build started...
Build succeeded.
The migration '20260402100808_Customer_Account_Relationship_Added' has already been applied to the database. Revert it and try again. If the migration has been applied to other
databases, consider reverting its changes using a new migration instead.
PM>
```

To remove a migration you must first update your database to a previous migration

You may update database to any previous Migration

```
PM> Update-Database 20260402095310_Customer_Added
Build started...
Build succeeded.
Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this takes too long.
Reverting migration '20260402100808_Customer_Account_Relationship_Added'.
Reverting migration '20260402100332_Customer_id_changed__Author_added'.
Done.
PM>
```

- Now we can remove the two migrations not applied on database